

ResearchAgent: An Autonomous AI Agent for Stock and Company Research

Project Proposal for UCS503P
Submitted to: Paramveer Ma'am
Thapar Institute of Engineering and Technology

Ananya Khanduja, CSED	Garv Sharma, CSED	Vishwa Choudhary, CSED
1024030693	1024030695	1024030668

August 25, 2026

1 Higher-order goal

This project aims to improve access to financial information and lower the barrier to informed investing by automating the labour-intensive process of company research. While the broader motivation is financial literacy and democratized access to research tooling, the immediate engineering objective is to translate *autonomous, tool-using AI agents* into a measurable, deployable software solution that produces a structured investment brief for any listed company in under a minute.

2 Time-to-value

- We will deliver meaningful increments quickly by prototyping the core agent loop (a single ticker, a single data source) and validating end-to-end report generation within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations and zero-config deployment targets (Render, for both the Go API service and the static React frontend) to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration – a working plan-act-synthesize loop for one company – then improved based on report-quality feedback.

3 Problem Statement

Individual investors and students researching a company or stock face a slow, fragmented process that discourages diligence. Common pain points include:

- **Fragmentation:** financials, news, and qualitative context live on separate sites (screeners, exchanges, news portals), forcing manual cross-referencing.
- **Cost barriers:** most automated research tools rely on paid financial-data or LLM APIs, which are inaccessible to students and retail investors.
- **Inconsistent structure:** self-compiled notes rarely follow a consistent, comparable format across companies.

- **Stale or incomplete data:** manually gathered snapshots quickly go out of date and often omit recent news or risk factors.

Why this matters now (contextual relevance): With free-tier LLMs and public data sources now capable of near-real-time synthesis, a zero-paid-API research agent can meaningfully close the access gap for individual users without institutional data subscriptions.

4 Proposed Solution

4.1 Overview

Build a **web-based** autonomous research agent, “ResearchAgent”, with the following components:

- **Company/ticker input portal:** user submits a company name or stock ticker (e.g. TCS, BLS).
- **Automated ticker resolution:** agent infers the correct NSE/BSE ticker symbol without manual lookup.
- **Autonomous data gathering:** agent pulls live financial data, web search results, and recent news for the target company.
- **Structured report synthesis:** agent compiles all gathered context into a consistent, six-section investment brief.
- **Report viewer:** a dedicated page renders the completed brief per ticker for review and sharing.

4.2 Core workflow

1. User submits a company name and the agent resolves it to a ticker symbol.
2. Agent plans a sequence of research steps (search, financials, news) tailored to that company.
3. Agent dispatches tool calls, observes results, and retries with a different query or tool if a source returns insufficient data.
4. Agent synthesizes all gathered context into the six-section report and renders it to the user.

4.3 Operational constraints for an educational, zero-budget setting

- Rely only on free-tier services: a single LLM API key (Gemini) is the sole required credential.
- Avoid dependence on paid financial-data providers; use a free-tier financial-data API (Twelve Data) as an optional primary source, with public scrapers and RSS feeds as layered fallbacks.
- Design for deployability using zero-config hosting (Render) for both the stateless Go backend and the static React frontend, avoiding any managed database or paid infrastructure.

5 Solution Approach

This is an engineering course project, so we focus on agent orchestration, tool-integration reliability, and system correctness rather than novel model research.

5.1 Agent orchestration

The core agent loop runs entirely inside a single API route and follows a *Plan* $\rightarrow$ *Act* $\rightarrow$ *Observe* $\rightarrow$ *Reflect* $\rightarrow$ *Synthesise* pattern:

- The LLM first generates a step-by-step research plan for the given company.
- The agent dispatches calls to three tools – search, financials, and news – based on that plan.
- After each tool call, the agent reflects on whether results are sufficient and retries with an alternate query or tool if not, without claiming guaranteed completeness.
- Gathered context from all tool calls is passed to the LLM for final synthesis into the report.

5.2 Web architecture

A decoupled Go backend and React frontend, communicating over a plain HTTP/JSON API:

- **Frontend:** a responsive UI (React with Vite and TypeScript, Tailwind CSS) for ticker input and report viewing, deployed as a static site independent of the backend.
- **Backend API:** a Go HTTP server, built on the standard library alone, exposing a core agent route (`/api/agent`) plus dedicated tool routes for search, financials, and news.
- **External data sources:** Twelve Data as an optional, documented primary source for financials, with a crumb-authenticated Yahoo Finance quote endpoint and a Screener.in scraper as layered fallbacks; DuckDuckGo HTML search with a Google News RSS fallback for web search; Google News RSS for dedicated news retrieval.

5.3 Search and data resilience

Because the system depends on unofficial/public sources, resilience is built in at the tool level:

- Web search attempts DuckDuckGo first, falls back to Google News RSS if rate-limited, and returns an empty result gracefully so the agent can retry rather than fail.
- Financial data resolution attempts Twelve Data first when configured (a single documented API covering both global and NSE/BSE-listed tickers), then falls back through a crumb-authenticated Yahoo Finance quote (with `.NS/.BO` suffix logic for Indian exchanges), an unauthenticated Yahoo chart endpoint, and finally a Screener.in scrape, returning whichever source succeeds first as a single financial-data object.

5.4 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build, vet, and test both the Go backend (`go build`, `go vet`, unit tests) and the React frontend (`tsc`, `vite build`) on every change.
- Produce repeatable deployment artifacts via Render’s zero-config pipelines for a Go web service and a static frontend site.
- Enable safe and frequent releases of incremental features (new report sections, new data sources) across both services independently.

6 Evaluation Criterion

We define success using measurable metrics tied to the core research pipeline.

6.1 Primary evaluation metric

Time-to-report (TTR): time between a user submitting a company name and receiving a complete, six-section report.

- Target: median TTR ≤ 60 seconds for a resolved NSE/BSE ticker.
- Attribution: measured server-side from request receipt to synthesis completion in the `/api/agent` route.

6.2 Secondary evaluation metrics

- **Ticker resolution accuracy:** percentage of company-name inputs correctly resolved to the right NSE/BSE ticker without manual correction.
- **Data completeness:** percentage of reports where all six sections are populated with non-empty, source-grounded content (no section left blank due to tool failure).
- **Fallback trigger rate:** frequency with which the search/news fallback path is invoked, as an indicator of primary-source reliability.
- **Reliability:** availability of the deployed agent endpoint during testing windows (e.g. $\geq 99\%$ uptime on Render free tier).

6.3 Pilot validation plan

- Run the agent against a curated list of 20–30 companies spanning large-cap, mid-cap, and small-cap NSE/BSE listings.
- Manually spot-check a sample of generated reports against source data (Screener.in, recent news) to validate factual grounding before broader use.

7 Scalability

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in concurrent analyses and request volume by:
 - decoupling the frontend from the stateless agent API route,
 - batching and caching tool responses (financials, news) per ticker to reduce redundant calls,
 - keeping each tool call independently retryable and timeout-bounded.
2. **Operationally deployable (practically):** The system should run entirely on free-tier infrastructure:
 - a stateless Go API service and a static React frontend, both hosted on Render’s free tier with no managed database to provision,
 - a single required LLM credential (Gemini, with OpenRouter as an optional backup path) and optional free-tier keys (Twelve Data, Exa) that gracefully degrade to public scrapers when absent,
 - CI/CD pipeline for automatic staging/production deployment of both services on every push.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- Go, using only its standard library, for the backend HTTP API; React with Vite and TypeScript for the frontend.
- Tailwind CSS v4 for a dark-mode-first responsive frontend.
- Google Gemini 2.0 Flash-Lite as the primary LLM, with an OpenRouter-routed Gemini 2.0 Flash wrapper as an optional fallback path.
- Twelve Data as an optional, documented free-tier financial-data API, with a crumb-authenticated Yahoo Finance quote endpoint and a Screener.in scraper as layered fallbacks, none requiring a paid API key.
- DuckDuckGo HTML search and Google News RSS for web search and news, with layered fallback, parsed using Go’s standard library.
- Render free tier for zero-config hosting and deployment of both the backend service and the static frontend.

We will use these mature components to focus on agent orchestration, data-source reliability, and report quality rather than inventing new infrastructure.

9 Project scope and deliverables

9.1 Initial deliverable

- Company/ticker input form and automated ticker resolution
- Core agent loop: plan generation, single-pass tool dispatch, and synthesis for one company at a time
- Financials tool (Twelve Data as optional primary source, with Yahoo Finance and Screener.in as layered fallbacks) and basic report rendering
- Basic logging and timestamps for time-to-report measurement
- CI: build + route-level tests + linting
- CD to Render staging with a single push-to-deploy pipeline

9.2 Subsequent deliverables

- Search and news tools with layered fallback (DuckDuckGo, Google News RSS)
- Reflect/retry logic so the agent re-queries when tool results are insufficient
- Full six-section report: overview, financial snapshot, news, competitive landscape, risk factors, and bull/bear investment summary
- Report history/viewer page per ticker and lightweight caching for repeat queries

10 Risks and mitigations

- **Unofficial data source instability:** Yahoo Finance’s quote endpoint and Screener.in are unofficial/scraped sources that can change or be blocked without notice; mitigate by preferring Twelve Data’s documented API when configured, and by layering an unauthenticated Yahoo chart fallback and graceful degradation rather than hard failures on the remaining unofficial paths.
- **Rate limiting:** DuckDuckGo scraping may be throttled under load; mitigate with the Google News RSS fallback and request pacing.
- **Hallucinated or unverifiable claims:** LLM synthesis may overstate confidence; mitigate by grounding synthesis strictly in retrieved tool output and clearly labelling the report as informational, not financial advice.
- **Free-tier quota limits:** Gemini/OpenRouter and Twelve Data free tiers cap request volume (e.g. Twelve Data: 800 requests/day, 8/minute); mitigate by using the OpenRouter fallback path, treating Twelve Data as an optional source with automatic fallback, and caching recent reports.

11 Summary

This project proposes a deployable, zero-paid-API platform – a Go backend paired with a React frontend – that autonomously researches a company or stock ticker and produces a structured investment brief. It meets the course criteria by emphasizing time-to-value through rapid, tool-by-tool iteration, implementing CI/CD to support frequent safe releases across both services, and using measurable evaluation criteria (especially time-to-report and data completeness). It remains focused on engineering agent orchestration and data-source reliability rather than research-driven model novelty.